Project Planning and Project Estimation Techniques

Employees in S/W industry at Managerial level are the most experienced persons. They often use different estimations techniques for proper planning to make optimum use of available resources during software development.

S. S. Satapathy

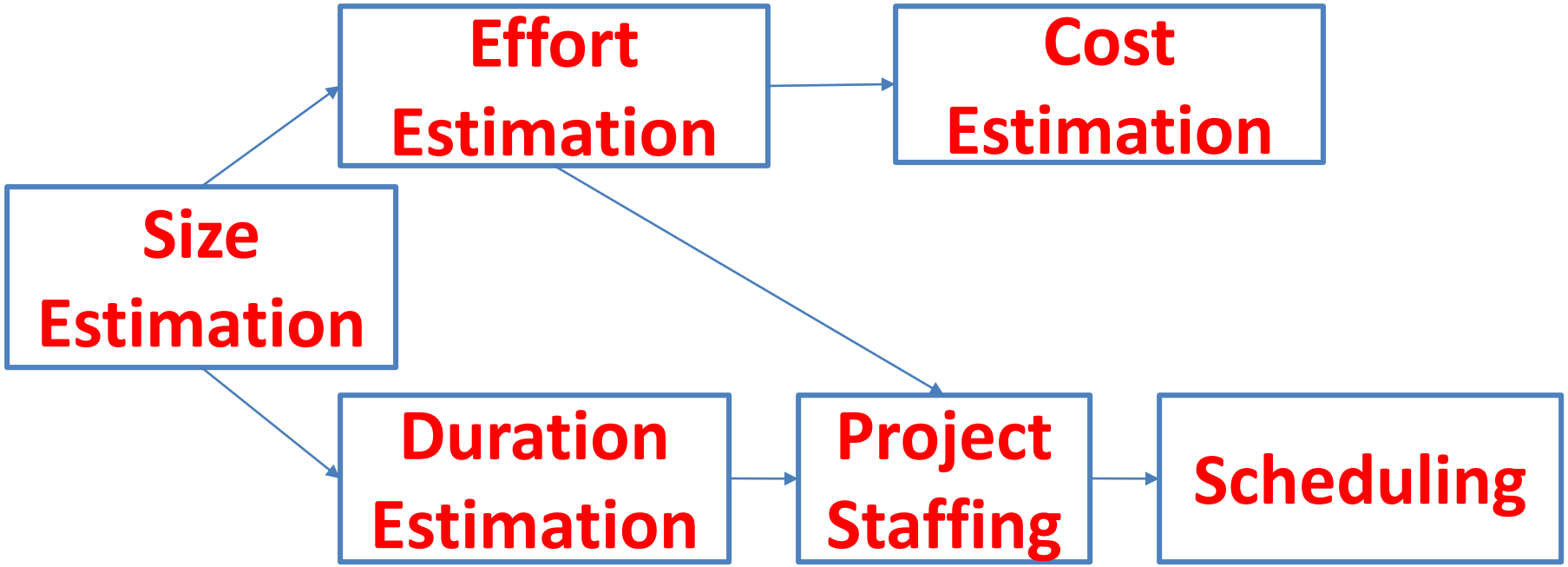
Responsibilities of a software project manager

- Software **project managers** take the overall responsibility for success of a project. **Manager** is responsible for project **planning**, and project **monitoring** and **control** activities.
- Some of the visible **job responsibility** of a **project manager**:
  - project proposal writing,
  - project cost estimation,
  - scheduling,
  - project staffing,
  - software process tailoring,
  - project monitoring and control,
  - software configuration management,
  - risk management,
  - interfacing with clients,
  - managerial report writing and presentations, etc.

Project planning

- **Once a project is found to be feasible**, software **project managers undertake project planning before starting of any development activity**. Project planning consists of:
  - Estimating the following attributes of the project
    - **Project size**: What will be problem complexity in terms of the effort and time required to develop the product?
    - **Cost**: How much is it going to cost to develop the project?
    - **Duration**: How long is it going to take to complete development?
    - **Effort**: How much effort would be required?
  - The subsequent planning activities such as **scheduling of manpower and other resources**, **staff organization**, **risk identifications**, miscellaneous plans are based on the accuracy of these estimations.

Precedence ordering among planning activities



Before proceeding for different estimations, **FIRST we need to have an idea of the product size** to develop. **Size estimation** (**Without looking at the product**) **is very difficult**. **But managers with sufficient experience in similar s/w development can make a fair guess**. Once size is estimated, other activities can be systematically completed.

Sliding Window Planning

- **Project planning** is a **very challenging** activity. **Improper planning** **can** even **cause project failure**.
- Especially for large projects, it is very much difficult to make accurate plans.
- A part of this difficulty is due to the fact that the proper parameters, scope of the project, project staff, etc. may change during the span of the project.
  - In order to overcome this problem, sometimes project managers undertake **project planning in stages**. Planning a project over a number of stages protects managers from making big commitments too early. This technique of staggered planning is known as **Sliding Window Planning**.
  - In this technique, starting with an initial plan, the project is planned more accurately in successive development stages.

Metrics for software project size estimation

- Accurate **estimation of the problem size is fundamental to satisfactory estimation of effort, time duration and cost of a software project**.
- In order to be able to accurately estimate the project size, some important metrics should be defined in terms of which the project size can be expressed.
  - The **size of a problem** is **neither** the **source code size** in terms of **number of bytes**, **nor** the **byte size** of the **executable code**.
  - The **project size is a measure of the problem complexity in terms of the effort and time required to develop the product**.
  - Two popular metrics used to estimate size are: **lines of code (LOC)** and **function point (FP)**

Lines of Code (LOC)

- LOC is the simplest and most popular metrics available to estimate project size.
- The **project size is estimated by counting the number of source instructions in the developed program**.
  - **Comment statements and header lines ignored in count**.
- Determining the **LOC count at the end** of a project **is simple**. However, **accurate estimation** of the **LOC count at the beginning** of a project **is very difficult**.
- In order to **estimate the LOC** count **at the beginning** of a project we need to:
  - **divide the problem into modules**, and each **module into sub-modules** and so on, until the sizes of the different **leaf-level modules can be approximately predicted**.
  - may be past experience in developing similar products will be helpful.
  - Estimate **total project size by combining estimation of the lowest level modules**

Shortcomings of LOC metric

- **LOC** gives a numerical value of **problem size** that can **vary widely with individual coding style**
  - For example, one programmer might write several source instructions on a single line whereas another might split a single instruction across several lines.
  - This problem may be overcome by counting the language tokens in the program rather than the lines of code.
- A good problem size **measure should consider the overall complexity of the problem and the effort needed** to solve it. That is, **it should consider** the local **effort needed to specify, design, code, test**, etc. and **not just the coding effort**.
- **LOC** measure **correlates poorly with the quality and efficiency of the code**. **Larger code size does not necessarily imply better quality or higher efficiency**.

Shortcomings of LOC

- **LOC** metric **penalizes use of higher-level programming languages, code reuse**, etc.
  - if a programmer uses several library routines, then the LOC count will be lower.
- **LOC** metric **measures the lexical complexity of a program and does not address the more important but subtle issues of logical or structural complexities**.
  - Between two programs with equal LOC count, a program having complex logic would require much more effort to develop than a program with very simple logic.

Shortcomings of LOC

- It is very **difficult to accurately estimate LOC in the final product from the problem specification**.
  - The LOC count can be accurately computed only after the code has been fully developed.
- Therefore, the **LOC metric is of little use to the project managers during project planning**, since project planning is carried out even before any development activity has started.
  - This possibly is the **biggest shortcoming of the LOC metric from the project manager's perspective**.

Function point (FP) proposed by Albrecht in 1983

- This metric overcomes many of the shortcomings of the LOC metric.
- One **important advantages** of using **FP** metric is that it **can be used to easily estimate the size of a software product directly from the problem specification**, whereas the size can be accurately determined only after the product has fully been developed in LOC count.
- The **conceptual idea behind the FP metric** is that **the size of a software product is directly dependent on the number of different functions or features it supports and technical issues in implementation**.

Function point (FP) proposed by Albrecht in 1983

- Broadly **software development** can be viewed as development of **five features** such as **(i)** **input** components, **(ii)** **output** components, **(iii)** **query** components, **(iv)** **Files/Database** components and **(v)** **User interfaces**.
- **Size of the software can be estimated** depending upon amount of these features to be developed.
- Depending on the **difficulty** in developing these features **weights** are associated with these features.

Function point (FP) proposed by Albrecht in 1983

- The size **of a product** in function points (**FP**) can be expressed as the **weighted sum** of **five problem characteristics: (Number of inputs), (Number of outputs), (Number of inquiries), (Number of files), (Number of interfaces)**
- **Function point computation is a two step process**.
- The **first step** is to compute the **unadjusted function point (UFP)** using formula:

  UFP = (Number of inputs)*4 + (Number of outputs)*5 + (Number of inquiries)*4 + (Number of files)*10 + (Number of interfaces)*10

  – The **weights associated** with the five characteristics were **proposed empirically** and **validated by the observations over many projects**.

Refinement in weights assigned to Function point (FP) entities

Type	Simple	Average	Complex
inputs	3	4	6
outputs	4	5	7
inquiries	3	4	6
files	7	10	15
interfaces	5	7	10

These weights can vary among software. If we can categories software in to three types, simple, average and complex, then weight associated with different software components are in order simple<average<complex.

Function point (FP) proposed by Albrecht in 1983

- Once the unadjusted function point (UFP) is computed, the **technical complexity factor (TCF)** is computed next **in 2nd step**.
- **TCF** refines the UFP measure by considering **fourteen other factors** such as high **transaction rates**, **throughput**, and **response time** requirements, **etc**. **Each of these 14 factors is assigned from 0 (not present or no influence) to 5 (strong influence)**. The resulting numbers are summed, yielding the total **degree of influence (DI)**.
- Now, **TCF is computed as (0.65+0.01*DI)**. As **DI can vary from 0 to 70, TCF can vary from 0.65 to 1.35**.
- **Finally, FP=UFP*TCF**.

Function point (FP) proposed by Albrecht in 1983

- **Number of inputs:** Each data item input by the user is counted. Individual data items input by the user are not considered in the calculation of the number of inputs, but **a group of related inputs are considered as a single input**.
  - For example, while entering the data concerning an employee to an employee pay roll software; the data items name, age, sex, address, phone number, etc. are together considered as a single input. All these data items can be considered to be related, since they pertain to a single employee.

Function point (FP) proposed by Albrecht in 1983

- **Number of outputs:** The outputs considered refer to **reports** printed, **screen outputs**, **error messages** produced, etc. While outputting the number of outputs the individual data items within a report are not considered, but a set of related data items is counted as one input.
- **Number of inquiries:** Number of inquiries is the number of **distinct interactive queries** which can be made by the users. These inquiries are the user commands which require specific action by the system.
- **Number of files: Each logical file** is counted. A logical file means groups of **logically related data**. Thus, logical files can be **data structures** or **physical files**.
- **Number of interfaces:** Here the interfaces considered are the **interfaces used to exchange information** with other systems. Examples of such interfaces are **data files on tapes**, disks, **communication links** with other systems etc.

Shortcomings of Function point (FP)

- A **major shortcoming** of the function point measure is that it **does not take into account the algorithmic complexity of a software**.
  - That is, the **function point metric assumes** that the **effort required to design and develop any two functionalities of the system is the same**.
- **It only takes the number of functions** that the system supports into consideration **without distinguishing the difficulty level of developing** the various functionalities.
- To overcome this problem, **an extension of the function point metric** called **feature point metric** is proposed.

Feature point

- Feature point metric incorporates an extra parameter **algorithm complexity,** in function point measure
- This parameter ensures that the computed size using the feature point metric reflects the fact that the more is the complexity of a function, the greater is the effort required to develop it and therefore its size should be larger compared to simpler functions.